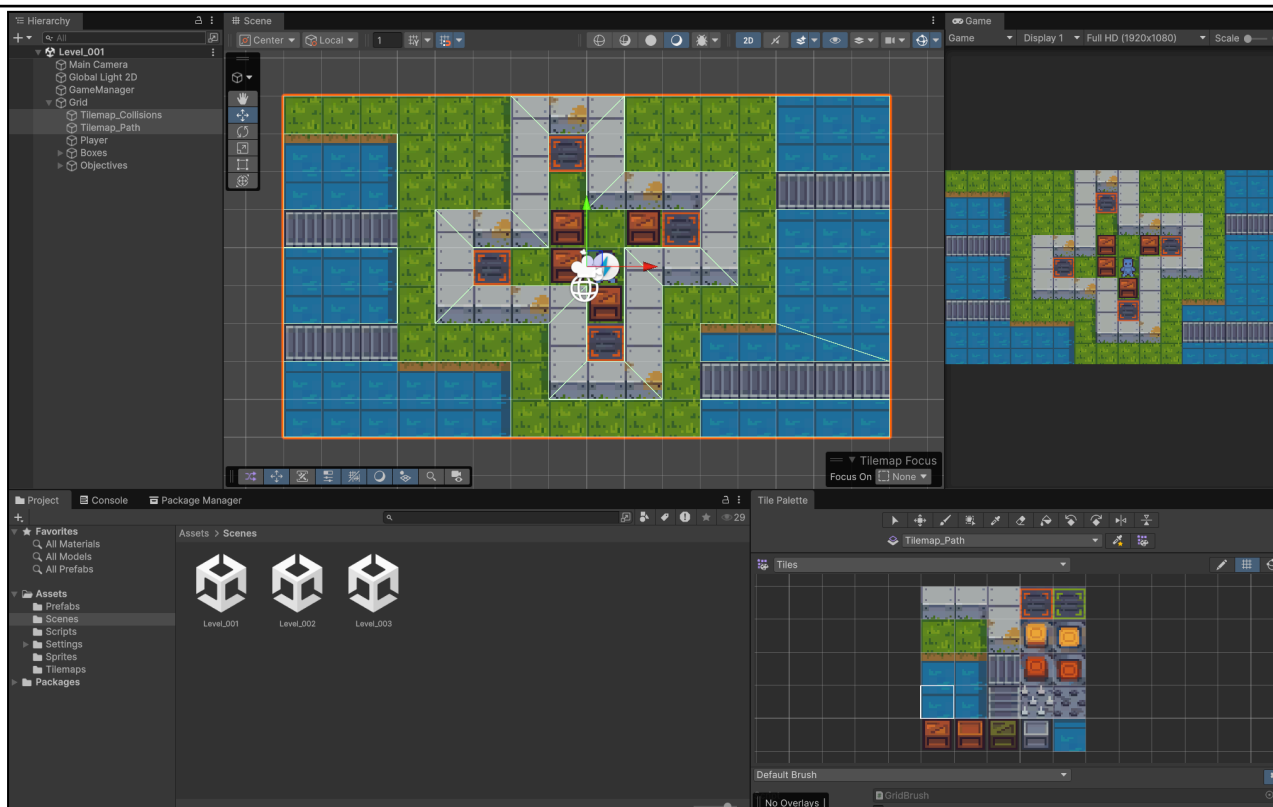


	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO 02

INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación Gráfica, Visión Computacional y Multimedia				
TÍTULO DE LA PRÁCTICA:	<i>Transformaciones Geométricas</i>				
NÚMERO DE PRÁCTICA:	02	AÑO LECTIVO:	2026	NRO. SEMESTRE:	IX
FECHA DE PRESENTACIÓN	03/05/26	HORA DE PRESENTACIÓN			
ESTUDIANTE:	Ponce de Leon Aguilar, Marco Antonio			NOTA:	
DOCENTE:	<i>Rene Alonso Nieto Valencia</i>				

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS Para la práctica de laboratorio se desarrolló un videojuego inspirado en el clásico Sokoban, en el cual se tendrán que atravesar los diferentes niveles colocando las cajas sobre los puntos objetivos dentro del mapa. El juego consta de 3 niveles con una dificultad progresiva en cada puzzle. 1. Creación del tilemap para los escenarios Se creó un tilemap con diferentes capas: ● Collisions, para objetos colisionables por los que no se puede pasar, como las paredes y el agua. ● Path, toda la zona por la que el jugador se puede desplazar, en este caso el pasto y los puentes. El jugador, las cajas y los objetivos se agregaron como objetos independientes, ya que cuentan con lógica adicional y se superponen sobre el tilemap, así que resulta más sencillo gestionarlos aparte. Los assets utilizados se descargaron de las siguientes fuentes: https://dani-maccari.itch.io/sokoban-tileset https://snoblin.itch.io/pixel-rpg-free-npc

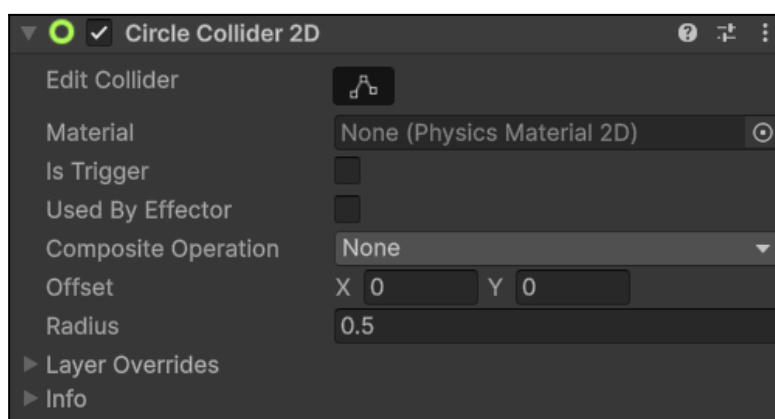


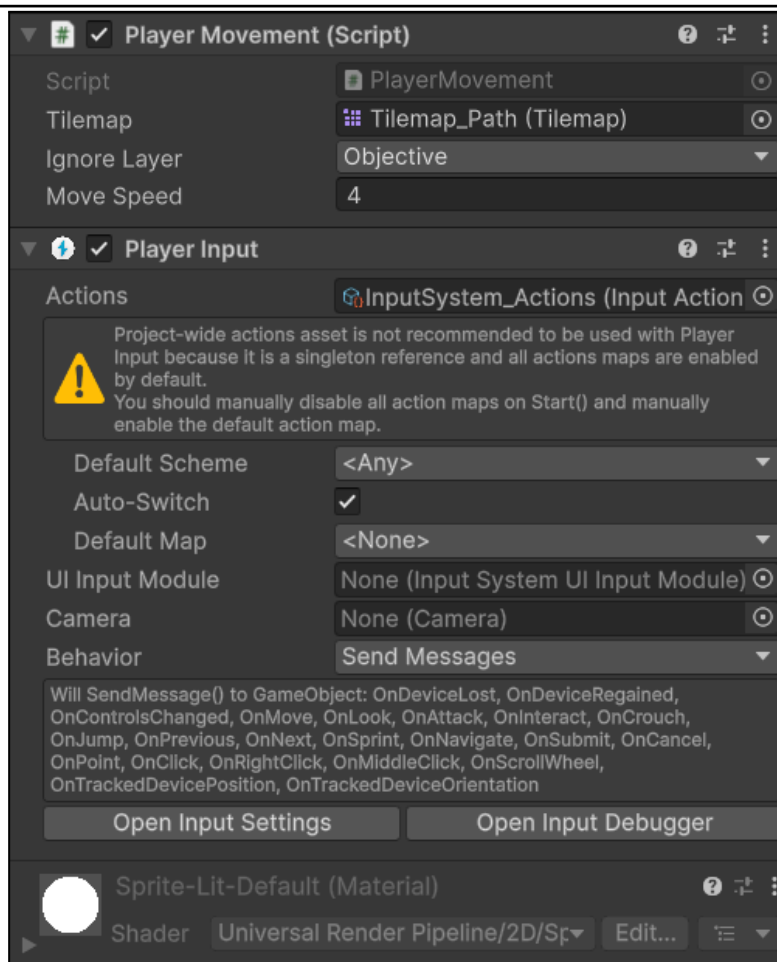
2. Movimiento del jugador

El movimiento del jugador está compuesto por un sistema de **Grid**, además de controlar que solamente se mueva a posiciones laterales y no diagonales en caso de presionar múltiples teclas de movimiento.

Para el movimiento en grilla es importante una referencia al **TileMap**, además de ignorar la colisión con los puntos objetivo, por lo que se tiene una variable para ignorar dicha capa en las colisiones.

Los componentes principales para el movimiento del jugador son los siguientes:





El código utilizado para el movimiento hace verificaciones en la casilla a la que el jugador quiere moverse, verificando que no haya un objeto en frente. En caso de existir algo, verifica si es una caja, para empujarla, y adicionalmente verifica que no haya nada obstruyendo el camino de la caja.

```
using UnityEngine;
using UnityEngine.InputSystem;
using UnityEngine.Tilemaps;

public class PlayerMovement : MonoBehaviour {
    public Tilemap tilemap;
    public LayerMask ignoreLayer;
    [SerializeField] private float moveSpeed = 4f;

    private PlayerInput playerInput;
    private InputAction moveAction;

    private Vector2Int gridPosition;
    private Vector3 targetPosition;
    private bool isMoving = false;
}
```

```
private void Awake() {
    playerInput = GetComponent<PlayerInput>();
    moveAction = playerInput.actions["Move"];
}

private void Start() {
    Vector3Int cell = tilemap.WorldToCell(transform.position);
    gridPosition = new Vector2Int(cell.x, cell.y);
    targetPosition = tilemap.GetCellCenterWorld(cell);
    transform.position = targetPosition;
}

private void Update() {
    if (!playerInput.enabled) return;
    HandleMovement();

    transform.position = Vector3.MoveTowards(transform.position, targetPosition, moveSpeed *
Time.deltaTime);

    if (transform.position == targetPosition) {
        isMoving = false;
    }
}

private void HandleMovement() {
    if (isMoving) return;

    Vector2 input = moveAction.ReadValue<Vector2>();
    if (input == Vector2.zero) return;

    Vector2 nextDirection = input;
    if (Mathf.Abs(nextDirection.x) > Mathf.Abs(nextDirection.y)) {
        nextDirection.y = 0;
    }
    else {
        nextDirection.x = 0;
    }
    nextDirection = nextDirection.normalized;
    Vector3 direction = new Vector3(nextDirection.x, nextDirection.y, 0);

    TryMove(direction);
}

private void TryMove(Vector3 direction) {
    Vector3 nextPosition = targetPosition + direction;
    Collider2D hit = Physics2D.OverlapCircle(nextPosition, 0.2f, ~ignoreLayer);

    if (hit == null) {
        MoveTo(nextPosition);
    }
    else if (hit.CompareTag("Box")) {
        Vector3 boxTarget = nextPosition + direction;
        Collider2D nextHit = Physics2D.OverlapCircle(boxTarget, 0.2f, ~ignoreLayer);
        if (nextHit == null) {
            hit.GetComponent<BoxMovement>().Move(direction);
            MoveTo(nextPosition);
        }
    }
}

private void MoveTo(Vector3 newPosition) {
    targetPosition = newPosition;
    isMoving = true;
}
}
```

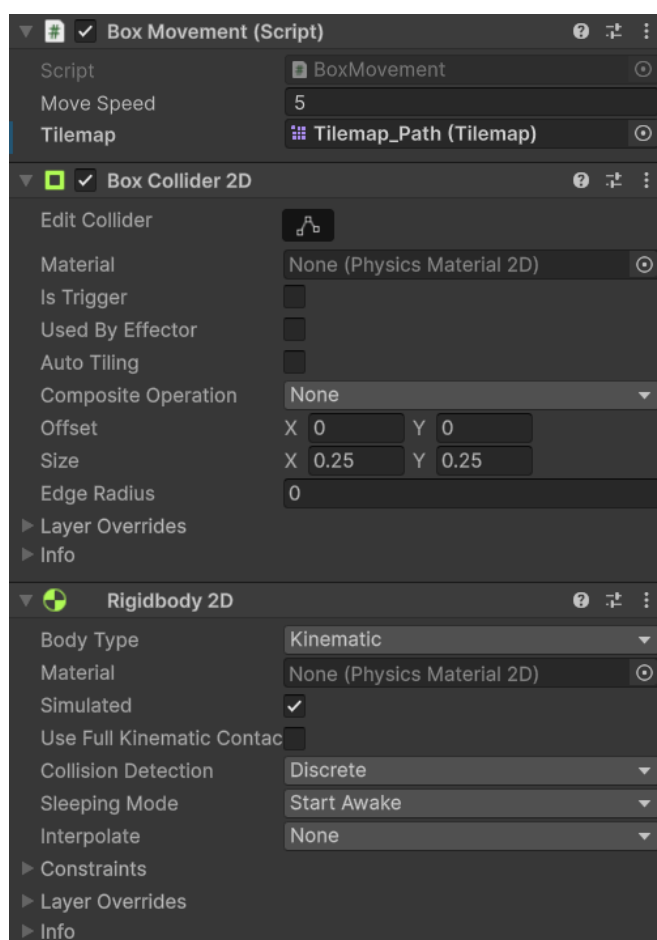
3. Movimiento de las cajas

Estas solamente pueden ser movidas por el jugador, cuando este llama a la función **Move** de **BoxMovement** desde **PlayerMovement** (se puede ver en la imagen anterior).

También requiere movimiento de grilla y cuenta con una velocidad de movimiento mayor a la del jugador para llegar a su destino antes y evitar inconsistencias.

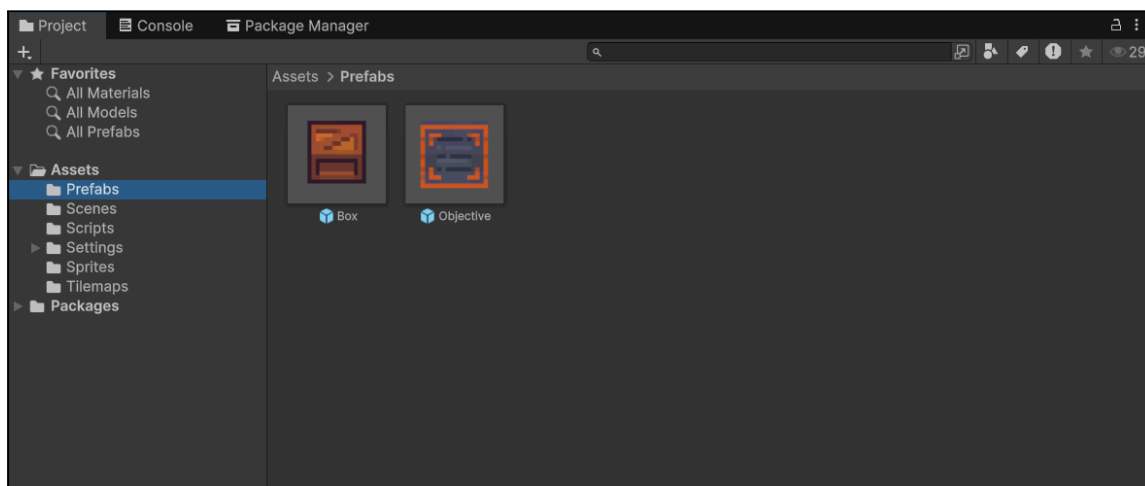
Además, el collider de la caja es relativamente pequeño para no detectar múltiples colisiones en ciertos casos.

Los componentes principales de una caja son los siguientes:



	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

Se creó un prefab para las cajas, así como para los objetivos, cuya lógica se verá más adelante.



El código utilizado para el movimiento sigue una lógica muy similar a la del movimiento del jugador.

```

using UnityEngine;
using UnityEngine.Tilemaps;

public class BoxMovement : MonoBehaviour {
    [SerializeField] private float moveSpeed = 4f;
    [SerializeField] private Tilemap tilemap;

    private Vector2Int gridPosition;
    private Vector3 targetPosition;
    private bool isMoving = false;

    private void Start() {
        Vector3Int cell = tilemap.WorldToCell(transform.position);
        gridPosition = new Vector2Int(cell.x, cell.y);
        targetPosition = tilemap.GetCellCenterWorld(cell);
        transform.position = targetPosition;
    }

    private void Update() {
        if (isMoving) {
            transform.position = Vector3.MoveTowards(transform.position, targetPosition, moveSpeed * Time.deltaTime);
            if (transform.position == targetPosition) {
                isMoving = false;
            }
        }
    }

    public void Move(Vector3 direction) {
        if (!isMoving) {
            targetPosition += direction;
            isMoving = true;
        }
    }
}

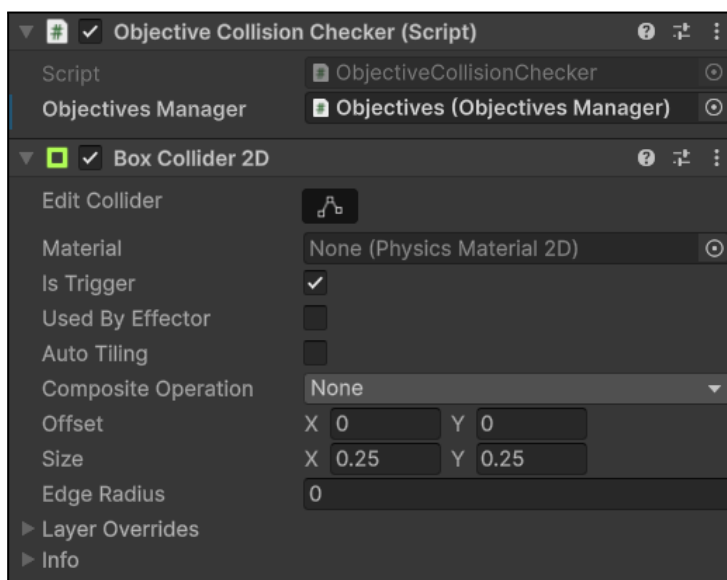
```

4. Detección de cajas en el objetivo

Para esto tenemos el prefab de **Objective** que indica dónde deben ubicarse las cajas para poder ganar.

En este caso también tenemos colliders pequeños para evitar colisiones múltiples, y como se mencionó antes, su capa de colisión es ignorada cuando el jugador quiere moverse y mover las cajas, para que no cuente como un obstáculo en el mapa. Se marca como trigger ya que no es un obstáculo de todas formas.

Sus componentes principales son un script para revisar las colisiones con las cajas y su respectivo collider trigger.



El código utilizado aquí se encarga de verificar cuando una caja entró en su collider y también cuando una caja sale del collider. Esto es para enviar señales a un gestor de los objetivos (**ObjectivesManager**) y así sumar o restar puntos según cuántas cajas hayan llegado a su objetivo.

Se considera la posibilidad de restar puntos porque una caja podría llegar a un objetivo de manera temporal, porque podría ser requerido cambiar de casilla en movimientos posteriores.

```
using UnityEngine;

public class ObjectiveCollisionChecker : MonoBehaviour {
    public ObjectivesManager objectivesManager;

    private void OnTriggerEnter2D(Collider2D collider)
    {
        if (collider.CompareTag("Box")) {
            objectivesManager.AddPoint();
        }
    }

    private void OnTriggerExit2D(Collider2D collider) {
        if (collider.CompareTag("Box")) {
            objectivesManager.RemovePoint();
        }
    }
}
```

Como se mencionó, se envían señales a un **ObjectivesManager**, en cuyo código se cuenta con una variable del puntaje a conseguir, que es la cantidad de cajas que se deben posicionar en los objetivos indicados.

```
using UnityEngine;

public class ObjectivesManager : MonoBehaviour {
    [SerializeField] private int pointsObjective = 0;

    private GameManager gameManager;
    private int points = 0;

    private void Start() {
        gameManager = GameObject.FindWithTag("GameManager").GetComponent<GameManager>();
    }

    public void AddPoint() {
        points++;
        if(points >= pointsObjective) {
            gameManager.Win();
        }
    }

    public void RemovePoint() {
        points = Mathf.Max(points - 1, 0);
    }
}
```


	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

5. GameManager

El **GameManager** se encarga de la lógica principal del juego, conteniendo el método **Win** que se llama desde **ObjectivesManager** para cargar el siguiente nivel con un cierto tiempo de espera para que no sea muy brusco (se carga el primer nivel cuando estamos en el último nivel).

Adicionalmente, aquí se gestionan los inputs para la tecla **R**, la cual reinicia un nivel en caso de haberse atascado, y de la tecla **Esc** para cerrar el juego.

```

using UnityEngine;
using UnityEngine.InputSystem;
using UnityEngine.SceneManagement;

public class GameManager : MonoBehaviour {
    private Keyboard keyboard;
    private const int MAX_LEVEL_BUILD_INDEX = 2;

    private void Awake() {
        keyboard = Keyboard.current;
    }

    private void Update() {
        if (keyboard.rKey.wasPressedThisFrame) {
            SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
        }
        if (keyboard.escapeKey.wasPressedThisFrame) {
            Application.Quit();
        }
    }

    public void Win() {
        int currentSceneIndex = SceneManager.GetActiveScene().buildIndex;

        if (currentSceneIndex == MAX_LEVEL_BUILD_INDEX) {
            Invoke(nameof(LoadFirstLevel), 1f);
        }
        else {
            Invoke(nameof(LoadNextLevel), 1f);
        }
    }

    private void LoadFirstLevel() {
        SceneManager.LoadScene(0);
    }

    private void LoadNextLevel() {
        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex +
1); }
}

```

6. Enlace al video de demostración

<https://drive.google.com/file/d/1btSqzI7hpXUQIDiYcttzxvEtV9gOJqfZ/view?usp=sharing>

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

RETROALIMENTACIÓN GENERAL